OS Remaining Questions — ★★and ★Answers

Short but Effective Format | Based on Silberschatz 10th Edition

---

CHAPTER 1: INTRODUCTION

★★"An operating system is similar to a government." - Justify.

Justification: - **Government role:** Provides framework for orderly functioning, doesn't perform useful work itself - **OS role:** Provides environment for programs to execute, doesn't do useful work directly - **Resource management:** Government allocates public resources (roads, utilities); OS allocates computer resources (CPU, memory, I/O) - **No monopoly on useful work:** Government doesn't produce goods; OS doesn't perform computations - **Protection and regulation:** Government enforces laws; OS enforces policies and prevents misuse - **Arbitration:** Government resolves disputes; OS resolves resource conflicts between processes

Both are **overhead** but necessary for organized, fair, and efficient operation.

★★User Mode to Kernel Mode Transition. Protection Mechanism.

Transition Process:

User Mode → Kernel Mode: 1. User program executes **system call** or **trap instruction** 2. CPU switches **mode bit** from 1 (user) to 0 (kernel) 3. Control transfers to OS at predetermined memory location 4. OS executes privileged operation 5. Mode bit switches back to 1 (user) 6. Control returns to user program

Triggers: - System call (voluntary) - Interrupt (hardware) - Exception/trap (error like division by zero)

Protection Mechanism:

Dual-mode operation protects OS and users from each other:

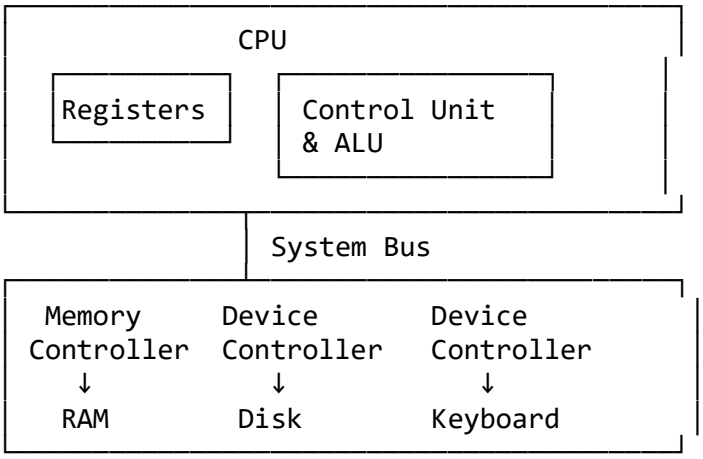
User Mode (Mode bit = 1)	Kernel Mode (Mode bit = 0)
Cannot execute privileged instructions	Can execute all instructions
Cannot access OS memory	Can access all memory
Cannot disable interrupts	Can disable interrupts
Cannot access I/O directly	Can access all I/O devices

Protection provided: - **Memory protection:** User cannot corrupt OS or other processes - **CPU protection:** Timer prevents infinite loops from monopolizing CPU - **I/O protection:** Prevents user from issuing illegal I/O commands - **System integrity:** Malicious/buggy programs cannot crash system

---

★General organization of computer system and role of interrupts

Computer System Organization:



Components: 1. **CPU:** Executes instructions 2. **Memory:** Stores data and instructions 3. **I/O devices:** Disks, keyboard, monitor, network 4. **Device controllers:** Each device has its own controller 5. **System bus:** Connects components

Role of Interrupts:

Interrupt: Signal to CPU that an event needs attention.

Types: - **Hardware interrupt:** From I/O device (disk read complete, key pressed) - **Software interrupt (trap):** From executing program (system call, error)

Interrupt-driven I/O: 1. CPU starts I/O operation, continues other work 2. Device controller completes operation, sends interrupt 3. CPU saves current state, executes interrupt handler 4. Handler processes I/O result 5. CPU resumes previous work

Benefits: - **Efficiency:** CPU doesn't wait idle for I/O - **Asynchronous:** Multiple operations happen concurrently - **Responsiveness:** System reacts to events immediately

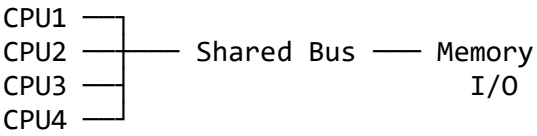
---

★Components in modern multiprocessor system

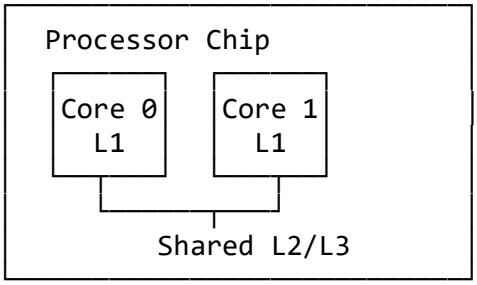
Multiprocessor system: Contains two or more processors sharing computer bus, memory, and peripherals.

Types:

1. Symmetric Multiprocessing (SMP): - All processors are **peers** (no master-slave) - Each processor runs identical copy of OS - Share memory and I/O devices - Example: Modern Intel/AMD multi-core CPUs



2. Multicore Processors: - Multiple CPU cores on **single chip** - Faster communication than separate processors - Shared cache levels (L2/L3)



Components: - **Multiple processors/cores:** Execute instructions in parallel - **Shared memory:** Common address space - **Interconnection network:** Connects processors (bus, crossbar, mesh) - **Cache hierarchy:** L1 (per core), L2/L3 (shared)

★Dual-mode operation

Dual-mode operation: Hardware provides at least **two modes** of operation.

Two Modes: 1. **User mode (mode bit = 1):** For user applications 2. **Kernel mode / Supervisor mode (mode bit = 0):** For OS

Mode bit: Hardware flag in CPU status register.

Purpose: - **Protection:** Distinguish OS execution from user execution - **Privileged instructions:** Only executable in kernel mode - **Safety:** Prevents user programs from damaging system

Examples of privileged instructions: - I/O control - Timer management - Interrupt management - Memory management instructions - Halt instruction

Switching: System call/interrupt → kernel mode; return → user mode.

★Timesharing vs Multiprogramming

Feature	Multiprogramming	Timesharing (Multitasking)
Goal	Maximize CPU utilization	Minimize response time
CPU switching	When process blocks (I/O)	Fixed time quantum (preemptive)
User interaction	Batch processing	Interactive
Response time	Not critical	Critical (milliseconds)
Context switch	On I/O wait	On timer interrupt
Example	Batch jobs on mainframe	Modern desktop OS

Key Difference: Timesharing is **preemptive** multiprogramming with **rapid switching** to create illusion of simultaneous execution for interactive users.

★★Open-source OS advantages and disadvantages

Advantages:

For developers/researchers: - Can study, modify, improve code - Learn OS internals - Fix bugs quickly - Customize for specific needs

For organizations: - No licensing fees - No vendor lock-in - Community support - Rapid security patches - Transparency (no hidden backdoors)

For innovation: - Peer review improves quality - Global collaboration - Faster development

Disadvantages:

For non-technical users: - Steeper learning curve - Less polished UI (sometimes) - Fragmentation (many distributions)

For companies: - No single accountable vendor - Support may be inconsistent - Compatibility issues with proprietary software - Harder to get professional training

For security-conscious: - Code visibility helps attackers too - Quality depends on community

---

★★Client-Server vs Peer-to-Peer distributed systems

Client-Server Model:

Structure: Centralized servers provide services to many clients.

```
    Server
   /  |  \
  C1  C2  C3  (Clients)
```

Characteristics: - Clear roles: server (provides), client (requests) - Server may be bottleneck - Easy to manage and secure - Examples: Web servers, email, databases

Peer-to-Peer (P2P) Model:

Structure: All nodes are equal (peers), each can be client and server.

```
P1 ↔ P2
↕     ↕
P3 ↔ P4
```

Characteristics: - No central server - More robust (no single point of failure) - Scalable (capacity grows with nodes) - Examples: BitTorrent, Skype, blockchain

P2P Applications:

- **File sharing:** BitTorrent, Napster
- **Communication:** Skype, Discord P2P voice
- **Distributed computing:** SETI@home
- **Cryptocurrency:** Bitcoin, Ethereum
- **Content delivery:** CDNs with P2P

★★Define OS. Services provided.

Operating System Definition:

OS = Program that acts as **intermediary** between user and hardware. Manages resources and provides execution environment.

Two views: 1. **User view:** Ease of use, convenience 2. **System view:** Resource allocator, control program

Services Provided:

1. User-Helpful Services:

Program execution: - Load program into memory - Run program - End execution (normal or abnormal)

I/O operations: - Read from disk, keyboard - Write to display, printer - Users can't access I/O directly (security)

File-system manipulation: - Create, delete, read, write files - Search files, manage permissions

Communications: - Inter-process communication (IPC) - Network communication

Error detection: - Monitor CPU, memory, I/O devices - Detect hardware/software errors - Take appropriate action

2. System-Efficiency Services:

Resource allocation: - Allocate CPU, memory, files, I/O among multiple processes - Scheduling algorithms

Protection and security: - Control access to resources - Authentication and authorization - Defend against attacks

Accounting: - Track resource usage - Billing in commercial systems

---

★Difficulties with read-only OS memory partition

Scheme: OS in read-only memory partition (neither user nor OS can modify).

Difficulties:

1. Cannot fix bugs or update OS: - Errors in OS remain permanent - Cannot patch security vulnerabilities - Cannot add new features - Would require hardware replacement

2. Cannot adapt to changing environment: - OS cannot store runtime data structures in its partition - Cannot maintain process tables, page tables - Cannot implement virtual memory - Limited to hardcoded configuration

Additional issues: - Inefficient use of memory - Cannot grow OS as needed - Debugging extremely difficult

---

★Clustered systems vs Multiprocessor systems

Multiprocessor Systems:

- Multiple CPUs in **single machine**
- Share common bus, memory, clock
- Tightly coupled
- Example: Dual-core laptop

Clustered Systems:

- Multiple **complete computers** (nodes) networked together
- Each node is independent computer
- Loosely coupled
- Share storage via SAN (Storage Area Network)
- Example: Google data centers

Requirements for High Availability Cluster:

Two machines must: 1. **Shared storage:** Both access same data (SAN) 2. **Heartbeat monitoring:** Constant health checks between nodes 3. **Failover mechanism:** Automatic service transfer if one fails 4. **State replication:** Active node mirrors state to standby 5. **Consistent cluster membership:** Agreement on which nodes are active

Example: Two database servers. If primary fails, standby detects via missed heartbeat and takes over within seconds.

★Mobile OS design challenges vs PC OS

Challenges:

1. Limited resources: - Less memory (2-8GB vs 16-64GB PC) - Slower processors (power constrained) - Limited storage

2. Power management: - Battery life critical - Aggressive power saving needed - Must manage CPU, display, wireless radios

3. Different I/O: - Touchscreen (no keyboard/mouse) - Sensors (GPS, accelerometer, gyroscope) - Limited screen size

4. Connectivity: - Intermittent network (WiFi/cellular switching) - Must handle disconnection gracefully - Background sync challenges

5. Security and privacy: - Personal data (contacts, location, photos) - Sandboxing apps crucial - Permission models

6. Real-time constraints: - Audio/video calls require low latency - Touch response must be immediate

---

★Multiprogramming vs Multiprocessing with examples

Feature	Multiprogramming	Multiprocessing
Definition	Multiple programs in memory, CPU switches between them	Multiple CPUs execute programs simultaneously
CPUs	Single CPU	Multiple CPUs

Feature	Multiprogramming	Multiprocessing
Concurrency	Pseudo-parallelism (interleaved)	True parallelism
Goal	Maximize CPU utilization	Increase throughput, speed
Example	Single-core CPU running Word + Chrome + Spotify (switches rapidly)	Quad-core CPU, each core running different process

Multiprogramming example: - CPU executes Process A - A waits for I/O → CPU executes Process B - B waits for I/O → CPU executes Process C - One CPU, never idle

Multiprocessing example: - CPU1 executes Process A - CPU2 executes Process B (at same time) - CPU3 executes Process C (at same time) - True simultaneous execution

---

★OS as resource allocator

Resource Allocator role: OS decides **how to allocate resources** to processes/users efficiently and fairly.

Resources managed: - **CPU time:** Which process gets CPU next? (Scheduling) - **Memory space:** How much RAM for each process? (Memory management) - **File storage:** Disk space allocation - **I/O devices:** Printer, network, disk access

Allocation decisions: - **Fairness:** All processes get reasonable share - **Efficiency:** Maximize utilization - **Priority:** Critical tasks get preference - **Deadlock avoidance:** Prevent circular resource waits

Example: 4 processes want CPU. OS scheduler decides: P1 runs 10ms, P2 runs 10ms, P3 runs 10ms, P4 runs 10ms, repeat. Each gets fair share.

★Why DMA (Direct Memory Access) is used

Without DMA (Programmed I/O): - CPU directly transfers each byte between I/O and memory - CPU wastes cycles moving data - Inefficient for large transfers (disk reads)

With DMA: - DMA controller handles data transfer - CPU initiates transfer, continues other work - DMA moves data directly: Device ↔ Memory (bypasses CPU) - Interrupt when complete

Advantages: 1. **CPU freed** from data transfer overhead 2. **Faster transfers** (no CPU involvement per byte) 3. **Better performance** for bulk I/O (disk, network)

Example: Reading 1MB file: - Without DMA: CPU moves 1 million bytes → wasted CPU time - With DMA: CPU says "DMA, move 1MB from disk to memory" → CPU does other work while DMA transfers

---

★Advantages of multiprocessor systems

1. Increased throughput: - More work done in less time - N processors ≈ N times speedup (not exactly due to overhead)

2. Economy of scale: - Sharing resources cheaper than N separate computers - Shared peripherals, power supply, housing

3. Increased reliability: - **Graceful degradation:** If one CPU fails, others continue (reduced performance, not total failure) - **Fault tolerance:** System continues despite failures

4. Better performance per cost: - Multiprocessor cheaper than multiple single-processor systems with same total power

---

CHAPTER 2: OS STRUCTURES

★OS in firmware vs disk

OS in Firmware (ROM/Flash):

Used in: - Embedded systems (routers, IoT devices) - Smartphones (bootloader) - BIOS/UEFI

Advantages: - Cannot be accidentally deleted - Boots faster (no disk read) - More secure (read-only)

Disadvantages: - Difficult/impossible to update - Limited space

OS on Disk:

Used in: - PCs, servers, workstations

Advantages: - Easy to update/upgrade - Large space available - Can have multiple OS (dual boot)

Disadvantages: - Disk failure → OS lost (needs reinstall) - Slower boot (read from disk) - Can be corrupted by malware

★System program

System programs: Provide convenient environment for program development and execution. Layer above OS, below applications.

Categories: 1. **File management:** cp, mv, rm, ls, mkdir 2. **Status information:** date, uptime, ps, df 3. **File modification:** Text editors (vi, nano, notepad) 4. **Programming support:** Compilers (gcc), debuggers (gdb), interpreters 5. **Communications:** Email, browsers, ssh 6. **Application programs:** Games, database systems

Difference from OS: - System programs **use** OS services (system calls) - OS **provides** fundamental services - System programs are **separate** processes

---

★Why use APIs instead of system calls directly

Advantages of APIs (like POSIX, Win32):

1. Portability: - Same API code works on different OS - Example: POSIX `open()` works on Linux, macOS, Solaris - Direct syscalls tie code to specific OS

2. Ease of use: - APIs provide higher-level abstractions - Handle complex details internally - Simpler parameter passing

3. Backward compatibility: - API remains stable across OS versions - System call numbers/interfaces may change

4. Additional functionality: - APIs may provide error checking, validation - Buffer management - Type safety

Example:

```
// Using API (stdio.h)
FILE *f = fopen("file.txt", "r");  // Simple, portable

// Direct system call (Linux)
int fd = syscall(SYS_open, "file.txt", O_RDONLY);  // OS-specific
```

★API vs ABI

Feature	API (Application Programming Interface)	ABI (Application Binary Interface)
Level	Source code level	Binary/machine code level
Defines	Function names, parameters, return types	Binary instruction format, calling conventions, register usage
Langua	High-level (C, Java)	Machine code

Feature	API (Application Programming Interface)	ABI (Application Binary Interface)
ge		
Portability	Source code portable (recompile needed)	Binary compatible (same arch)
Example	POSIX API: `read(fd, buf, count)`	x86-64 ABI: parameters in rdi, rsi, rdx registers

API: Programmer writes code using API → recompile for each platform
ABI: Binary executable runs directly (no recompile) if ABI matches

---

★Why applications are OS-specific

Reasons:

1. Different system calls: - Linux: `fork()`, Windows: `CreateProcess()` - APIs differ

2. Different executable formats: - Linux: ELF (Executable and Linkable Format) - Windows: PE (Portable Executable) - macOS: Mach-O

3. Different ABIs: - Calling conventions differ - Register usage differs

4. Different library dependencies: - Linux: `.so` shared libraries - Windows: `.dll` DLLs - Linked at different locations

Solutions: - **Virtual machines:** JVM, .NET (write once, run anywhere) - **Interpreted languages:** Python, JavaScript - **Cross-compilation:** Compile for target OS - **Compatibility layers:** Wine (run Windows apps on Linux)

★System calls to start new process on UNIX

Shell executes following system calls:

1. `fork()`: - Creates child process (duplicate of parent) - Child gets copy of parent's memory, registers

2. `exec()` family: - Replaces child's memory with new program - Variants: `execl()`, `execv()`, `execve()`, `execvp()`

3. `wait()` (if shell waits for completion): - Parent waits for child to finish

Example (shell executing `ls`):

```
pid_t pid = fork();           // Create child

if (pid == 0) {               // Child process
    execl("/bin/ls", "ls", NULL);  // Replace with ls
```

```
    exit(1);                    // Only if exec fails
}
else {                          // Parent (shell)
    wait(NULL);                 // Wait for ls to finish
}
```

★★Layered approach difficulty with dependencies

Problem: Components A and B depend on each other (circular dependency).

Example scenario:

Device Driver (Layer 3) needs Memory Manager (Layer 4): - Driver allocates buffers from memory manager

Memory Manager (Layer 4) needs Device Driver (Layer 3): - Memory manager may use paging → needs disk driver to swap pages

Circular dependency: Layer 3 ↔ Layer 4

Solutions: 1. **Careful redesign:** Break circular dependency by extracting common functionality into lower layer 2. **Relax layering:** Allow limited cross-layer calls 3. **Use different approach:** Microkernel or modular design (modern OSes avoid strict layering)

★★Microkernel advantages, interaction, disadvantages

Advantages:

1. Easier to extend: - Add services without modifying kernel - Just add new user-space server

2. More reliable: - Service crash doesn't crash kernel - Isolated failures

3. More secure: - Minimal kernel → smaller attack surface - Services run with limited privileges

4. Portable: - Small kernel easier to port to new hardware

User Programs ↔ Services Interaction:

Communication via message passing:

```
User Program → IPC message → Microkernel → IPC message → File Server
```

All communication goes **through microkernel** using IPC (Inter-Process Communication).

Disadvantages:

1. Performance overhead: - Message passing slower than function calls - Context switches expensive - Copy overhead (user space → kernel → user space)

2. Complexity: - Message-based communication more complex than direct calls - Harder to debug

---

★★Android and standard Java

Question: Why doesn't Android use standard Java API and JVM?

Reasons:

1. Performance: - Standard JVM designed for desktops - Too heavy for mobile (memory, power)

2. Android uses Dalvik/ART VM: - **Dalvik:** Register-based (faster on mobile ARM) - **ART (Android Runtime):** AOT (Ahead-Of-Time) compilation for speed - Standard JVM: Stack-based

3. Different bytecode format: - Standard Java: `.class` files (Java bytecode) - Android: `.dex` files (Dalvik Executable) → more compact

4. Mobile-specific APIs: - Android APIs for sensors, location, touch, app lifecycle - Standard Java APIs not suitable (designed for desktop/server)

5. Licensing: - Android uses open-source Apache Harmony class library - Avoids Oracle licensing issues

Process:

```
Java source → javac → .class files → dx → .dex file → ART
```

★Two categories of OS services

Category 1: User-Beneficial Services

Purpose: Help the user

- Program execution
- I/O operations
- File-system manipulation
- Communications
- Error detection

Category 2: System-Efficiency Services

Purpose: Ensure efficient operation

- Resource allocation
- Accounting (usage tracking)
- Protection and security

Difference: - **Category 1:** Directly visible/useful to user - **Category 2:** Background functions for system efficiency, may be invisible to user

---

★Five OS services creating convenience (impossibility for user-level)

1. Program Execution:

Convenience: User just clicks icon; OS loads, allocates memory, starts execution
Impossible at user level: Cannot access memory management hardware

2. I/O Operations:

Convenience: Simple `read()`/`write()` calls
Impossible: Cannot access I/O hardware directly (privileged instructions)

3. File-System Manipulation:

Convenience: Files organized hierarchically with names
Impossible: Cannot access raw disk sectors (protection violation)

4. Communications:

Convenience: Network send/receive simplified
Impossible: Network hardware access requires kernel mode

5. Error Detection:

Convenience: OS detects hardware errors, prevents crashes
Impossible: Monitoring CPU/memory errors requires privileged access

---

★How OS handles `open()` system call

Flow:

```
User Application
    |
    | open("file.txt", O_RDONLY)  // API call
    ▼
C Library (libc)
```

```
    |
    | Sets syscall number in register
    | TRAP instruction
    ▼
──────────────────────────────────────
  KERNEL MODE (Mode bit = 0)
──────────────────────────────────────
    |
System Call Interface
    |
    | Reads syscall number
    | Calls corresponding kernel function
    ▼
VFS (Virtual File System)
    |
    | Determines file system type
    ▼
Specific File System (ext4, NTFS, etc.)
    |
    | Checks permissions
    | Locates file on disk
    | Allocates file descriptor
    | Creates file table entry
    ▼
Return file descriptor
    |
──────────────────────────────────────
  USER MODE (Mode bit = 1)
──────────────────────────────────────
    |
    ▼
User Application receives fd
```

Steps: 1. User calls `open()` API

2. Library places syscall number in register, executes trap

3. CPU switches to kernel mode

4. Kernel validates parameters

5. VFS determines file system

6. File system checks permissions, finds file

7. Allocates file descriptor 8. Returns to user mode with fd

★System calls to copy file content

Sequence:

```c
// Open source file for reading
int src_fd = open("source.txt", O_RDONLY);

// Create destination file for writing
int dest_fd = creat("dest.txt", 0644);  // or open with O_CREAT|O_WRONLY

// Copy loop
char buffer[4096];
ssize_t bytes_read;

while ((bytes_read = read(src_fd, buffer, sizeof(buffer))) > 0) {
    write(dest_fd, buffer, bytes_read);
}

// Close files
close(src_fd);
close(dest_fd);
```

System calls used: 1. **`open()`:** Open source file 2. **`creat()` or `open()`:** Create destination file 3. **`read()`:** Read from source (in loop) 4. **`write()`:** Write to destination (in loop) 5. **`close()`:** Close both files (×2)

CHAPTER 3: PROCESSES

★★Process Control Block (PCB)

PCB (Process Descriptor): Data structure containing all information about a process.

Contents:

1. Process state: New, ready, running, waiting, terminated
2. Program counter: Address of next instruction
3. CPU registers: Accumulator, index registers, stack pointer, general-purpose registers
4. CPU-scheduling information: Priority, scheduling queue pointers
5. Memory-management information: Base/limit registers, page tables, segment tables
6. Accounting information: CPU time used, time limits, process ID
7. I/O status information: Open files, allocated I/O devices

Necessity:

1. Context switching: - Save process state when switching - Restore state when resuming

2. Scheduling: - Scheduler needs priority, state

3. Resource management: - Track resource usage and allocation

4. Protection: - Ensure process doesn't access unauthorized resources

Without PCB: OS couldn't multiplex CPU among processes — no multitasking possible.

★★Context switch actions

Context switch: CPU switches from one process to another.

Actions by Kernel:

1. Save state of old process: - Save CPU registers → PCB of old process - Save program counter - Save stack pointer

2. Update PCB of old process: - Change state (running → ready) - Update accounting info (CPU time)

3. Select new process: - Scheduler chooses from ready queue

4. Update PCB of new process: - Change state (ready → running)

5. Restore state of new process: - Load registers from PCB of new process - Load program counter - Load stack pointer - Switch page tables (memory context)

6. Resume execution: - Jump to program counter of new process

Time: 1-1000 microseconds (pure overhead)

★★Context switch as pure overhead - Justify

Context switch time = pure overhead because:

1. No useful work done: - System doesn't execute user code - System doesn't perform I/O - Only saves/restores state

2. CPU wasted: - Thousands of instructions just to switch - Could have been used for user computation

3. Cache pollution: - Old process's cache data becomes useless - New process suffers cold cache (slower access) - Cache warm-up overhead

4. TLB flush: - Translation Lookaside Buffer must be cleared (or tagged) - Future memory accesses slower initially

Example: - Context switch takes 5 microseconds - If switching every 10ms (time quantum) - Overhead = 5μs / 10,000μs = 0.05% → seems small - But with hundreds of switches/second → adds up

Tradeoff: More context switches → better responsiveness but more overhead.

★Cooperating process

Cooperating process: Process that can **affect or be affected by** other processes executing in the system.

Characteristics: - Shares data with other processes - Communicates with other processes (IPC)

Examples: - Producer-consumer (shared buffer) - Compiler pipeline: preprocessor → compiler → assembler → linker

Contrast with Independent Process: - Cannot affect or be affected by others - Doesn't share data - Example: Two separate text editors editing different files

Why cooperate: - Information sharing (multiple users access same data) - Computation speedup (parallel subtasks) - Modularity (divide system into separate functions) - Convenience (multi-tasking)

★★Process creation with `fork()`

`fork()` System Call:

Creates child process — duplicate of parent.

Return values: - Parent: receives child's PID (> 0) - Child: receives 0 - Error: returns -1

Example:

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {        // Error
        printf("Fork failed\n");
    }
    else if (pid == 0) {  // Child process
        printf("Child process (PID=%d)\n", getpid());
    }
    else {                // Parent process
        printf("Parent process (PID=%d, child PID=%d)\n", getpid(), pid);
    }

    return 0;
}
```

Output (order may vary):

```
Parent process (PID=1234, child PID=1235)
Child process (PID=1235)
```

How many times "Operating System" printed:

```
fork();
fork();
printf("Operating System\n");
```

Answer: 4 times

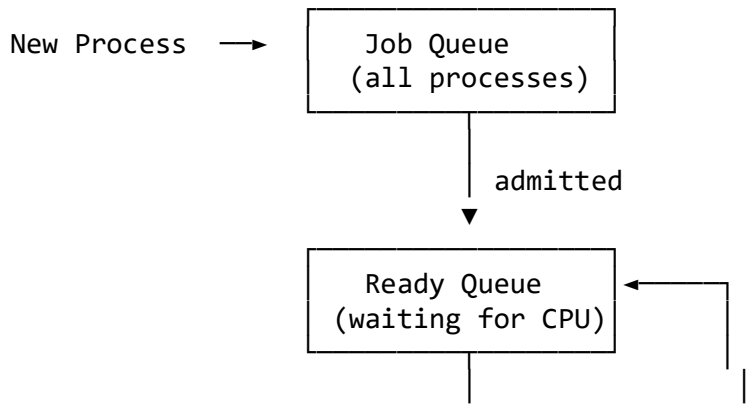
Explanation: - 1st `fork()`: Creates 1 child → 2 processes - 2nd `fork()`: Both execute it → 4 processes total - All 4 print → 4 outputs

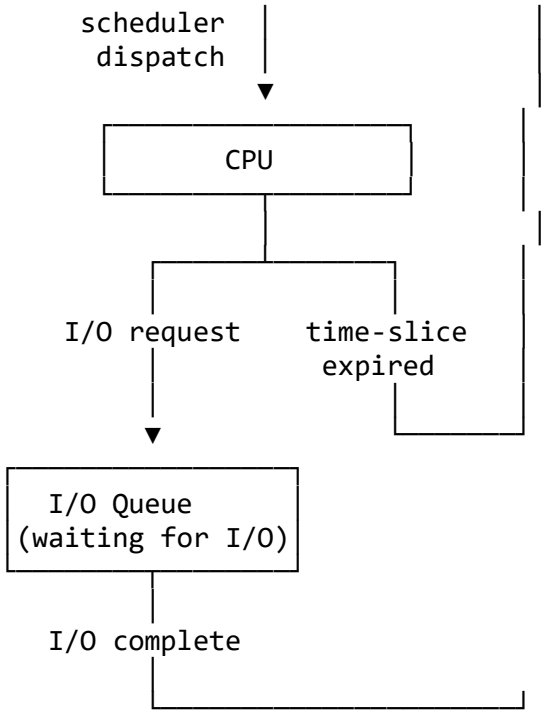
```
        P0
       /  \
   fork()  fork()
    /        \
   P1         (P0 continues)
  /  \        /  \
fork()     fork()
 /  \       /    \
P2  P3    P0'    P1'
```

```
Total: P0, P0', P1, P1', P2, P3 (but actually P0 = P0', etc., so 4 unique)
Simpler: 2^n processes where n = number of forks
2^2 = 4 processes
```

★★Process scheduling queuing diagram

Queuing diagram shows how processes move through system queues.





Queues: 1. **Job queue:** All processes in system 2. **Ready queue:** Processes ready to execute (waiting for CPU) 3. **Device queues:** Processes waiting for I/O devices

Flow: - New → Ready → Running → (I/O wait → I/O queue → Ready) → ... → Terminate

---

★★Remote Procedure Call (RPC). IPC comparison.

RPC Definition:

RPC: Allows a program to **execute a procedure on a remote computer** as if it were local.

Mechanism: 1. Client calls procedure (looks like local call) 2. **Stub** packages parameters (marshaling) 3. Message sent over network 4. Server stub unpacks parameters 5. Server executes procedure 6. Result sent back 7. Client receives result

Transparency: Client doesn't know procedure is remote.

IPC Comparison (covered in ★★★- see Ch3 Q8 in first file)

★Process creation and termination

Process Creation:

Methods: 1. **System initialization:** OS creates initial processes at boot 2. **User request:** User clicks program icon 3. **`fork()`:** Process creates child

Steps: 1. Assign unique PID 2. Allocate memory 3. Initialize PCB 4. Load program into memory 5. Set PC to entry point 6. Place in ready queue

Process Termination:

Normal termination: - `exit()` system call - Returns status to parent

Abnormal termination: - Error (divide by zero) - `kill()` signal - Parent terminates (cascading termination)

Steps: 1. Deallocate memory 2. Close open files 3. Remove from all queues 4. Delete PCB

★Process creation in UNIX with example

See ★★answer above (`fork()` system call).

Additional example with `exec()`:

```
pid_t pid = fork();

if (pid == 0) {  // Child
    execlp("/bin/ls", "ls", "-l", NULL);
    // If exec succeeds, this line never executes
}
else {  // Parent
    wait(NULL);  // Wait for child
}
```

Child replaces its image with `ls` program.

★IPC models strengths and weaknesses

See ★★★Chapter 3 Q8 for full comparison.

Summary:

Shared Memory: - **Strengths:** Fast, efficient for large data - **Weaknesses:** Synchronization difficult, same-machine only

Message Passing: - **Strengths:** Easy synchronization, works across network - **Weaknesses:** Slower (system call overhead), less efficient for bulk data

---

★Ordinary pipes vs Named pipes

Ordinary (Anonymous) Pipes:

Characteristics: - One-way communication (half-duplex) - Only between parent-child processes - Exist only while processes run - Created: `pipe()` system call

Suitable for: - Shell pipelines: `ls | grep txt` - Parent-child communication - Temporary data passing

Named Pipes (FIFOs):

Characteristics: - Bidirectional possible - **Any processes** can communicate (no parent-child requirement) - Persist as filesystem object (even after creating process exits) - Created: `mkfifo()` or `mkfifo` command

Suitable for: - Unrelated processes communication - Client-server on same machine - Persistent communication channel

Example use cases:

Ordinary: `gcc file.c -o prog | more` (compiler to pager)
Named: Database server uses named pipe for client connections

---

★Benefits/disadvantages of IPC design choices

1. Synchronous vs Asynchronous:

Synchronous (blocking): - **Benefit:** Automatic synchronization, simpler logic - **Disadvantage:** Sender/receiver blocked, potential deadlock

Asynchronous (non-blocking): - **Benefit:** No blocking, better performance - **Disadvantage:** Complex buffering, race conditions

2. Automatic vs Explicit Buffering:

Automatic (OS-managed): - **Benefit:** Programmer doesn't handle buffers - **Disadvantage:** Unpredictable buffer size, potential overflow

Explicit (user-managed): - **Benefit:** Full control, predictable behavior - **Disadvantage:** More complex code

3. Send by Copy vs Send by Reference:

Copy: - **Benefit:** Safe (receiver can't corrupt sender's data), works across network - **Disadvantage:** Slower (copy overhead)

Reference (pointer): - **Benefit:** Fast (no copy) - **Disadvantage:** Only same address space, data corruption risk

4. Fixed-size vs Variable-size Messages:

Fixed-size: - **Benefit:** Simple implementation, predictable - **Disadvantage:** Wastes space (small messages), can't send large data

Variable-size: - **Benefit:** Flexible, efficient space use - **Disadvantage:** Complex buffer management

---

CHAPTER 4: THREADS & CONCURRENCY

★Resources used for thread vs process creation

Thread Creation:

Resources allocated: - Thread ID - Program counter - Register set - Stack

Resources shared (NOT created): - Code section - Data section - Heap - Open files - Signals

Cost: Low (only stack allocation)

Process Creation:

Resources allocated: - Everything a thread needs PLUS: - Separate address space (code, data, heap) - New PCB - Page tables - File descriptor table (copy)

Cost: High (complete duplication)

Comparison: - Thread creation: ~30× faster than process (Solaris) - Thread context switch: faster (no memory context switch)

★User-level vs Kernel-level threads

Two Key Differences:

1. Kernel awareness: - **User threads:** Kernel doesn't know they exist (managed by thread library) - **Kernel threads:** Kernel directly schedules them

2. Blocking behavior: - **User threads:** If one blocks (system call), entire process blocks - **Kernel threads:** If one blocks, others continue

Table:

Feature	User Threads	Kernel Threads
Created by	Thread library	OS kernel
Scheduled by	Application	Kernel scheduler
Blocking	Whole process blocks	Only that thread blocks
Context switch	Fast (no kernel)	Slower (kernel mode)
Multicore use	No (mapped to 1 kernel thread)	Yes (true parallelism)

★Data parallelism vs Task parallelism

Data Parallelism:

Same operation on **different subsets of data** in parallel.

Example:

```
// Sum array elements in parallel
Thread 1: sums A[0..249]
Thread 2: sums A[250..499]
Thread 3: sums A[500..749]
Thread 4: sums A[750..999]
```

All threads run **same code** (addition) on different data.

Task Parallelism:

Different operations (tasks) performed in parallel.

Example:

```
Thread 1: Reads data from network
Thread 2: Processes data (computation)
Thread 3: Renders UI
Thread 4: Writes results to disk
```

Each thread runs **different code**.

★★Amdahl's Law + calculation

Amdahl's Law:

Predicts **speedup** from parallel processing.

Formula:

```
Speedup = 1 / (S + (1-S)/N)
```

```
where:
S = fraction that must be serial (0 to 1)
N = number of processing cores
1-S = parallel fraction
```

Implication: Speedup limited by serial portion.

Calculation (60% parallel, 2 and 4 cores):

Given: - Parallel component = 60% = 0.6 - Serial component S = 1 - 0.6 = 0.4

For N=2 cores:

```
Speedup = 1 / (0.4 + 0.6/2)
        = 1 / (0.4 + 0.3)
        = 1 / 0.7
        = 1.43
```

For N=4 cores:

```
Speedup = 1 / (0.4 + 0.6/4)
        = 1 / (0.4 + 0.15)
        = 1 / 0.55
        = 1.82
```

Conclusion: Doubling cores (2→4) doesn't double speedup due to serial portion limiting parallelism.

★★Multithreading models

Three Models:

1. Many-to-One:

```
User Threads:  T1  T2  T3  T4
                \   |   |  /
                 \  |  | /
Kernel Thread:    KT1
```

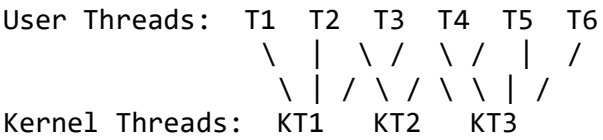
- Many user threads → 1 kernel thread
- **Advantage:** Fast context switch
- **Disadvantage:** One blocks → all block; can't use multicore
- **Example:** Green threads

2. One-to-One:

```
User Threads:   T1    T2    T3    T4
                 |     |     |     |
Kernel Threads: KT1   KT2   KT3   KT4
```

- Each user thread → own kernel thread
- **Advantage:** True parallelism; one blocks → others continue
- **Disadvantage:** Overhead (kernel threads expensive)
- **Example:** Linux, Windows

3. Many-to-Many:



- Many user threads → many (but fewer) kernel threads
- **Advantage:** Flexibility; some concurrency without too much overhead
- **Disadvantage:** Complex to implement
- **Example:** Solaris prior to version 9

★★Java Threads. Implicit threading approaches.

Java Threads:

Creation:

```java
// Method 1: Extend Thread class
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
new MyThread().start();

// Method 2: Implement Runnable
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread running");
    }
}
new Thread(new MyRunnable()).start();
```

Implicit Threading:

Programmer specifies tasks, not threads directly.

1. Thread Pools:

Concept: Pre-create threads, reuse for tasks.

```java
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.execute(task1);
```

```
pool.execute(task2);
pool.shutdown();
```

Benefits: - Faster (no thread creation overhead per task) - Limits number of threads (prevents resource exhaustion)

2. Fork-Join:

Divide-and-conquer parallelism.

```
if (task small enough) {
    solve directly
}
else {
    split into subtasks
    fork subtasks (create threads)
    join results (wait for completion)
}
```

Example: Parallel merge sort

3. Grand Central Dispatch (GCD):

Apple's technology (macOS, iOS).

Concept: Submit blocks/closures to dispatch queues.

```
dispatch_queue_t queue = dispatch_get_global_queue(...);
dispatch_async(queue, ^{
    // Code block runs on queue
});
```

Serial vs Concurrent queues: - Serial: Tasks execute one at a time (FIFO) - Concurrent: Tasks execute in parallel

---

★Multithreaded better than single-threaded on single-processor

When multiple kernel threads help on single CPU:

1. I/O-bound application: - Thread 1 blocks on I/O → Thread 2 runs → CPU not idle - Example: Web server waiting for disk reads

2. Hiding latency: - One thread waits for memory/cache → another runs - Better CPU utilization

3. Responsiveness: - UI thread remains responsive while background thread computes

Not better when: - CPU-bound tasks with no I/O - Overhead > benefits

---

★Four CPUs: How many I/O threads? How many CPU-intensive threads?

Scenario: 2 dual-core processors = 4 CPUs. Application does I/O and CPU-intensive work.

Answer:

I/O threads: More than 4 (e.g., 10-20) - I/O threads mostly blocked (waiting for disk, network) - While one waits, others can run - Doesn't waste CPU since blocked threads don't consume CPU

CPU-intensive threads: 4 (one per CPU) - Each CPU runs one thread continuously - More than 4 → context switching overhead, no benefit

Reasoning: - CPU-bound: threads compete for CPU → limit to number of CPUs - I/O-bound: threads wait → can have many (overlap I/O wait with computation)

★Complications of concurrent processing (iOS example)

Three major complications:

1. Synchronization: - Race conditions on shared data - Need locks, semaphores → complexity - Potential deadlocks

2. Difficult debugging: - Non-deterministic behavior (timing-dependent bugs) - Hard to reproduce - Example: Bug appears only when two threads execute in specific order

3. Resource management: - More threads → more memory (stacks) - Context-switching overhead - Must balance concurrency vs overhead

iOS initially avoided these by using single-threaded model (simpler, more predictable). Later added Grand Central Dispatch for controlled concurrency.

★Concurrency without parallelism - is it possible?

Yes, concurrency ≠ parallelism.

Definitions:

Concurrency: Multiple tasks **in progress** (interleaved execution)
Parallelism: Multiple tasks **executing simultaneously** (same instant)

Example: Single-core CPU

```
Time:  0    1    2    3    4    5
CPU:   [T1] [T2] [T1] [T3] [T2] [T1]
```

- **Concurrent:** T1, T2, T3 all in progress (interleaved)
- **NOT parallel:** Only one executes at any instant

Concurrency creates illusion of parallelism through rapid switching.

True parallelism requires: Multiple CPUs/cores

CHAPTER 5: CPU SCHEDULING

★★Why CPU scheduling needed? Basis for scheduling.

Why Needed:

1. Maximize CPU utilization: - CPU should not sit idle when processes wait for I/O - Switch to another process

2. Multiprogramming: - Multiple processes compete for CPU - Need policy to decide who gets CPU

3. Fairness: - All processes should get reasonable CPU time

4. Interactive responsiveness: - Users expect quick response times

Basis for Scheduling:

Observation: Most processes alternate between: - **CPU burst:** Period of computation - **I/O burst:** Waiting for I/O

```
Process execution: [CPU burst] → [I/O burst] → [CPU burst] → ...
```

Decision points (when scheduling occurs): 1. Process switches from **running → waiting** (I/O request) 2. Process switches from **running → ready** (interrupt, time quantum expired) 3. Process switches from **waiting → ready** (I/O complete) 4. Process **terminates**

Preemptive: Scheduling at points 2, 3
Nonpreemptive: Only at points 1, 4

★★Types of schedulers

1. Long-term Scheduler (Job Scheduler):

Selects: Which processes admitted to ready queue from job pool
Frequency: Infrequent (minutes)
Controls: Degree of multiprogramming
Goal: Good mix of I/O-bound and CPU-bound processes

Not present in modern PC/mobile OS (all programs admitted immediately).

2. Short-term Scheduler (CPU Scheduler):

Selects: Which process from ready queue gets CPU next
Frequency: Very frequent (milliseconds)
Must be fast: Invoked often, must have low overhead
Algorithms: FCFS, SJF, Priority, RR

3. Medium-term Scheduler (Swapper):

Purpose: Swap processes out of memory to disk (and back)
Goal: Reduce degree of multiprogramming
Used when: Memory pressure (too many processes)

★Dispatcher

Dispatcher: Module that gives CPU control to the process selected by scheduler.

Functions: 1. **Context switch:** Switch to selected process 2. **Switch to user mode:** Change from kernel to user mode 3. **Jump to proper location:** Resume user program at saved PC

Dispatch latency: Time to stop one process and start another (overhead).

Goal: Minimize dispatch latency (microseconds).

★Starvation (Indefinite postponement)

Starvation: Process waits **indefinitely** because other processes continuously get preference.

Causes: - **Priority scheduling:** Low-priority process never executes if high-priority keep arriving - **Resource allocation:** Process never gets needed resource

Example: - Process P priority 10 (low) - High-priority processes (50, 60) keep arriving - P never gets CPU → starves

Solution: - **Aging:** Gradually increase priority of waiting processes - After time t, increase priority by 1 - Eventually, even low-priority process has high priority

---

★★Multiprocessor and multicore scheduling issues

Issues:

1. Load balancing: - Distribute processes evenly across CPUs - **Push migration:** Periodic task moves processes from overloaded to idle CPU - **Pull migration:** Idle CPU pulls waiting task

2. Processor affinity: - Process prefers to stay on same CPU (warm cache) - **Soft affinity:** Try to keep on same CPU, but can migrate - **Hard affinity:** Never migrate - **Tradeoff:** Affinity vs load balancing

3. NUMA (Non-Uniform Memory Access): - Some memory closer to certain CPUs - Schedule process on CPU near its memory for speed

4. Multicore processors: - **Memory stall:** CPU waits for memory (cache miss) - **Hyperthreading/SMT:** Two hardware threads per core - While one thread stalls, switch to other → utilize core better

5. Heterogeneous multiprocessing: - **big.LITTLE:** Fast cores (big) + slow/power-efficient cores (LITTLE) - Schedule intensive tasks on big, background on LITTLE - Example: ARM mobile processors

---

★Scheduling criteria conflicts

1. CPU Utilization vs Response Time:

Conflict: Maximizing utilization → long time quantum → slower response.

- High utilization: Keep CPU busy with long processes
- Low response: Quick switches for interactive tasks
- **Tradeoff:** Long quantum (high util, poor response) vs short quantum (low util, good response)

2. Average Turnaround Time vs Maximum Waiting Time:

Conflict: Minimizing average TAT may increase max wait.

- SJF minimizes average TAT but long jobs wait very long (high max wait)
- FCFS gives fair max wait but poor average TAT
- **Tradeoff:** Fairness vs efficiency

3. I/O Utilization vs CPU Utilization:

Conflict: Optimizing for CPU-bound processes neglects I/O devices.

- CPU utilization favors CPU-bound processes
- I/O utilization favors I/O-bound processes

- **Tradeoff:** Balance workload mix

★Smaller time quantum increases context switches

Time quantum (q): Time slice for Round Robin.

Example:

Process P: burst time = 10ms

Case 1: q = 10ms: - P runs 0-10ms, finishes - **Context switches:** 1 (when done)

Case 2: q = 1ms: - P runs: 0-1, 1-2, 2-3, ..., 9-10 - **Context switches:** 10

Smaller q → more switches → more overhead.

Rule of thumb: 80% of CPU bursts should be shorter than q.

★Exponential averaging for CPU burst prediction

Formula: Predict next burst based on past:

```
τ(n+1) = α × t(n) + (1-α) × τ(n)

where:
τ(n+1) = predicted next burst
t(n) = actual last burst
τ(n) = previous prediction
α = weighting factor (0 ≤ α ≤ 1), typically 0.5
```

Example (α = 0.5):

History: t(1)=6, t(2)=4, t(3)=6, t(4)=4
Initial: τ(1) = 10

```
τ(2) = 0.5×6 + 0.5×10 = 3 + 5 = 8
τ(3) = 0.5×4 + 0.5×8 = 2 + 4 = 6
τ(4) = 0.5×6 + 0.5×6 = 3 + 3 = 6
τ(5) = 0.5×4 + 0.5×6 = 2 + 3 = 5
```

Sequence of estimates: 10, 8, 6, 6, 5

★Relation between parameterized scheduling algorithms

Priority and FCFS:

FCFS = Priority scheduling where priority = arrival time

- Process arriving first has highest priority
- No preemption

RR and SJF:

SJF = RR with q = ∞

- If time quantum is infinite, each process runs to completion
- Equivalent to SJF (shortest runs first, completes first)

★RR variant with PCB pointers in ready queue

Normal RR: Queue contains processes
Variant: Queue contains **pointers to PCBs**

Effect:

Problem: If a process is both in ready queue and I/O queue, there are **duplicate pointers**.

When process finishes I/O: - Moves to ready queue (new pointer) - **Two pointers** to same PCB in ready queue

Effect: Process gets CPU **twice as often**

Advantages:

Faster response for I/O-bound processes (they get CPU more often after I/O).

Disadvantages:

Unfair: I/O-bound processes get CPU more frequently.

Fix:

Search ready queue before adding pointer. If PCB already present, don't add duplicate.

OR: Use process ID list instead of pointers; check for duplicates.

CHAPTER 6: SYNCHRONIZATION TOOLS

★Busy waiting

Busy waiting (spinlock): Process **continuously checks** a condition in a loop while waiting.

```
while (lock == 1)
    ;  // Do nothing, just loop
lock = 0;  // Acquire lock
```

Characteristics: - Process **wastes CPU cycles** checking repeatedly - No productive work during wait

When acceptable: - **Short waits** on multiprocessor systems - One CPU spins while another releases lock quickly

Problem: - **Wastes CPU** that could be used by other processes - **Inefficient** on single-CPU systems

Alternative: Blocking: Process sleeps, wakes when lock available (no CPU waste).

★★Mutex lock solution to critical section

Mutex lock provides mutual exclusion.

Code:

```
// Shared variable
mutex lock;  // Initially unlocked

// Process structure
do {
    acquire(&lock);      // Entry section

        /* CRITICAL SECTION */

    release(&lock);      // Exit section

    /* REMAINDER SECTION */
} while (true);
```

acquire() implementation:

```
void acquire(mutex *lock) {
    while (!atomic_compare_and_swap(&lock->locked, 0, 1))
        ;  // Busy wait
}
```

release() implementation:

```
void release(mutex *lock) {
    lock->locked = 0;  // Atomic store
}
```

Properties: - ✓ Mutual exclusion (atomic acquire/release) - ✓ Progress (lock eventually available) - ✓ Bounded waiting (in some implementations using queues)

Hardware support needed: Atomic operations (test-and-set, compare-and-swap).

★★Producer-Consumer problem with critical section

Problem:

Producer creates data, puts in buffer.
Consumer takes data from buffer.

Shared buffer:

```
int buffer[N];
int count = 0;  // Number of items
```

Critical Section:

Producer:

```
while (true) {
    // Produce item

    while (count == N)
        ;  // Wait if buffer full

    // CRITICAL SECTION START
    buffer[in] = item;
    in = (in + 1) % N;
    count++;
    // CRITICAL SECTION END
}
```

Consumer:

```
while (true) {
    while (count == 0)
        ;  // Wait if buffer empty

    // CRITICAL SECTION START
    item = buffer[out];
    out = (out + 1) % N;
    count--;
    // CRITICAL SECTION END

    // Consume item
}
```

Where is critical section? - Reading/writing `count` - Accessing shared buffer

Problem: `count++` and `count--` not atomic → race condition!

Solution: Use semaphore or mutex to protect critical section.

★★Monitor - high-level synchronization construct

Monitor: High-level abstraction providing **mutual exclusion automatically**.

Structure:

```
monitor MonitorName {
    // Shared variables

    // Procedures (only one executes at a time)
    procedure P1(...) { ... }
    procedure P2(...) { ... }

    // Condition variables
    condition x, y;

    // Initialization
    { initialization code }
}
```

Key features:

1. Automatic mutual exclusion: - Only **one process** can be active in monitor at any time - No need for explicit locks

2. Condition variables: - **x.wait():** Suspend process, release monitor lock - **x.signal():** Wake one waiting process

Advantage over semaphores: - **Encapsulation:** Data + procedures together - **Compiler enforces** mutual exclusion - **Less error-prone:** Can't forget to acquire/release

Example (Producer-Consumer with monitor):

```
monitor ProducerConsumer {
    condition full, empty;
    int count = 0;

    procedure insert(item) {
        if (count == N) full.wait();
        // Insert item
        count++;
        empty.signal();
    }

    procedure remove() {
        if (count == 0) empty.wait();
        // Remove item
```

```
        count--;
        full.signal();
    }
}
```

★Modifying semaphore wait() and signal() to overcome problems

Standard semaphore problem: Busy-waiting wastes CPU.

Solution: Blocking semaphores using wait queues.

Modified Definitions:

Semaphore structure:

```
typedef struct {
    int value;
    process *queue;  // Waiting processes
} semaphore;
```

wait() with blocking:

```
void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        // Add process to S->queue
        block();  // Sleep, yield CPU
    }
}
```

signal() with wakeup:

```
void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        // Remove process P from S->queue
        wakeup(P);  // Make P ready
    }
}
```

Improvement: - **No busy-waiting:** Process sleeps instead of looping - **CPU available** for other processes - **Context switch overhead:** But better than wasting CPU

CHAPTER 7: SYNCHRONIZATION EXAMPLES

★★Readers-Writers problem

Problem:

Database shared by: - **Readers:** Only read data (multiple simultaneous OK) - **Writers:** Modify data (exclusive access needed)

Constraints: 1. Multiple readers can read simultaneously 2. Only one writer can write at a time 3. No readers when writer is writing

First Readers-Writers Problem:

No reader should wait unless a writer is writing.

Shared variables:

```
semaphore rw_mutex = 1;  // Writer mutual exclusion
semaphore mutex = 1;     // Protect read_count
int read_count = 0;      // Number of readers
```

Writer:

```
wait(rw_mutex);
    // Write
signal(rw_mutex);
```

Reader:

```
wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);  // First reader locks out writers
signal(mutex);

    // Read

wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);  // Last reader unlocks
signal(mutex);
```

Problem: Writers may starve (readers continuously arrive).

Second Readers-Writers Problem:

Writers get priority: Once writer arrives, no new readers start.

★★Readers-Writers solution with semaphores

See above. Full code with binary semaphores (value 0 or 1).

mutex semaphore: Protects `read_count`
rw_mutex semaphore: Provides exclusive write access

CHAPTER 8: DEADLOCKS

★★Methods for handling deadlock. Deadlock avoidance.

Four Methods:

1. Deadlock Prevention: - Ensure at least one of 4 necessary conditions never holds - **Deny mutual exclusion, hold-and-wait, no preemption, or circular wait**

2. Deadlock Avoidance: - Allow conditions to hold but avoid unsafe states - Requires advance information (max resource needs) - **Banker's Algorithm**

3. Deadlock Detection and Recovery: - Allow deadlock to occur - Detect it (detection algorithm) - Recover (kill processes, preempt resources)

4. Ignore (Ostrich Algorithm): - Pretend deadlocks never happen - Used by most OS (Windows, Linux) — rare enough to ignore - Cheaper than prevention/avoidance

Deadlock Avoidance:

Concept: System decides whether granting a request would lead to **unsafe state**.

Safe state: System can allocate resources in some order and avoid deadlock.

Banker's Algorithm: - Before granting request, simulate allocation - Run safety algorithm - If safe → grant; if unsafe → deny (process waits)

Requires: Processes declare **max resource needs** in advance.

★★Resource-allocation graph deadlock analysis

See ★★★Chapter 8 Q10 for full diagrams.

Simple example:

Deadlock graph:

```
P1 → R1 → P2 → R2 → P1  (cycle)
```

Single instance per resource → **DEADLOCK**

★★Banker's Algorithm problem

See ★★★Chapter 8 Q11 for full algorithm and example.

Variant: 12 tape drives, 3 processes.

Given allocation and max needs, determine if safe.

Apply safety algorithm to check if all processes can finish.

★Cycle necessary but not sufficient for deadlock - Justify

Statement: Cycle in resource-allocation graph is **necessary** but **not sufficient** for deadlock.

Necessary:

If deadlock exists → cycle must exist.

Proof: If P0 waits for P1, P1 for P2, ..., Pn for P0 → forms cycle.

Not Sufficient:

Cycle exists → deadlock **may or may not** exist.

When cycle but NO deadlock: - **Multiple instances** of resource type - Resource instance available elsewhere

Example:

```
P1 requests R1 (has 2 instances)
P2 holds one instance of R1, requests R2
P3 holds R2, requests R1
Cycle: P1→R1→P2→R2→P3→R1 (back to P1)

But P1 can get second instance of R1 (not held by anyone)
→ P1 finishes → releases R1 → P2 can get it
→ No deadlock despite cycle!
```

Conclusion: Cycle + single instance = deadlock. Cycle + multiple instances = maybe deadlock.

★Safe, unsafe, and deadlocked states

Safe State:

System can allocate resources to each process in some order and avoid deadlock.

Exists a safe sequence <P1, P2, ..., Pn> where for each Pi: - Resources Pi needs ≤ Available + (Held by all Pj where j < i)

Example: - Available = (3,3,2) - Process needs satisfiable in order P1, P3, P4, P0, P2 - **SAFE**

Unsafe State:

No safe sequence exists — but deadlock not guaranteed yet.

May lead to deadlock if processes request max resources.

Example: - Available = (1,0,0) - No process can finish with current available - If all request max → deadlock - **UNSAFE** (but currently not deadlocked)

Deadlocked State:

Processes permanently blocked, waiting in circular chain.

All deadlocked states are unsafe.
Not all unsafe states are deadlocked (may become deadlocked).

Venn diagram concept:

```
All States
  └─ Safe States (no deadlock possible)
  └─ Unsafe States
       └─ Deadlocked States (deadlock occurred)
```

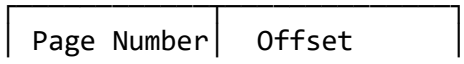
CHAPTER 9: MAIN MEMORY

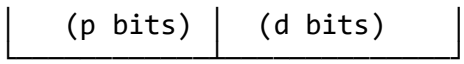
★★Page sizes are always power of 2 - Justify

Reason: Simplifies address translation using **bit shifting**.

Address Structure:

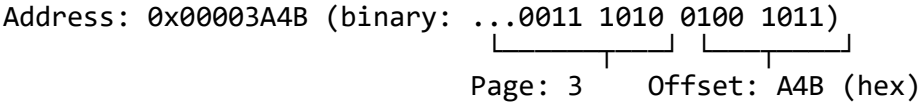
Logical address (n bits):





If page size = 2^d: - Offset = **lower d bits** of address - Page number = **upper (n-d) bits**

Example: 32-bit address, page size = 4KB = 2^12



No division/modulo needed: - Page number = Address >> 12 - Offset = Address & 0xFFF

If page size NOT power of 2 (e.g., 6KB): - Page number = Address / 6144 (slow division) - Offset = Address % 6144 (slow modulo)

Hardware efficiency: Bit operations (shift, mask) much faster than division/modulo.

★★Internal vs External fragmentation

Internal Fragmentation:

Wasted space INSIDE allocated memory block.

Cause: Memory allocated in **fixed-size units** (pages/blocks) larger than needed.

Example (paging): - Page size = 4KB - Process needs 10.5KB - Allocated 3 pages (12KB) - **Waste: 1.5KB** inside last page

Average waste: 0.5 × page size per process

External Fragmentation:

Wasted space BETWEEN allocated blocks.

Cause: Free memory exists but is **scattered** in small non-contiguous holes.

Example (variable partitioning):

```
Memory: [P1: 50KB] [Free: 10KB] [P2: 30KB] [Free: 15KB]
Request: 20KB
→ Cannot satisfy (10+15=25 total, but not contiguous)
```

Total free: 25KB, but fragmented → unusable

Comparison:

Type	Location	Cause	Solution
Internal	Inside blocks	Fixed-size allocation	Smaller page/block size
External	Between blocks	Variable	Compaction, Paging

Type	Location	Cause	Solution
		allocation	

Paging eliminates external but causes internal fragmentation.

★Context switching time high for swapping - Justify

Swapping: Move entire process between memory and disk.

Why slow:

1. Disk I/O is slow: - Memory: ~10-100 ns - Disk: ~5-10 ms (50,000× slower)

2. Large data transfer: - Swap out: Write entire process to disk (MBs) - Swap in: Read entire process from disk

3. Context switch includes: - Save process state (fast) - **Swap out current process** (SLOW) - **Swap in next process** (SLOW) - Restore state (fast)

Example:

Process size: 100 MB
Disk transfer rate: 50 MB/s
Swap out time: 100/50 = 2 seconds
Swap in time: 2 seconds
Total context switch: ~4 seconds (extremely high!)

Modern systems: Avoid full swapping. Use **paging** (swap individual pages, not entire process).

★★Paging example: 32-byte memory, 4-byte pages

See ★★★Chapter 9 Q14 for complete example with translation.

Summary: - 32 bytes / 4 bytes per page = 8 frames - Logical address 2 bits page, 2 bits offset - Page table maps logical pages to physical frames

---

★★TLB advantage and usage in paging

Translation Lookaside Buffer (TLB):

Hardware cache storing recent page table entries.

Without TLB:

Every memory access requires **2 memory accesses:** 1. Read page table (translate address)
2. Read actual data

Slow!

With TLB:

TLB hit: Page number → frame number (no page table access)
TLB miss: Access page table, update TLB

Effective Access Time:

```
EAT = Hit_Ratio × (TLB_time + Mem_time)
    + (1 - Hit_Ratio) × (TLB_time + 2 × Mem_time)
```

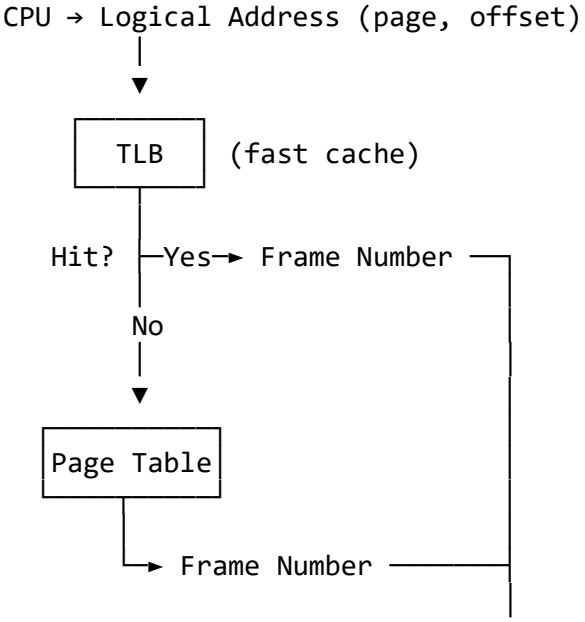
Example (from book):

TLB time: 20 ns
Memory time: 100 ns
Hit ratio: 80%

```
EAT = 0.8 × (20 + 100) + 0.2 × (20 + 200)
    = 0.8 × 120 + 0.2 × 220
    = 96 + 44
    = 140 ns
```

Without TLB: 200 ns
With TLB: 140 ns → **30% faster**

TLB Use Diagram:



```
▼
(Frame, Offset) → Physical Address
```

★Ways to manage memory for CPU/response improvement

Techniques:

1. Swapping: - Move processes between memory and disk - Free memory for active processes

2. Multiprogramming: - Keep multiple processes in memory - CPU switches when one blocks

3. Paging: - Divide memory into fixed-size pages - Eliminate external fragmentation - Allow non-contiguous allocation

4. Virtual Memory: - Execute partially-loaded programs - Demand paging (load pages only when needed) - Programs larger than physical memory

5. Caching: - Cache frequently accessed data - L1, L2, L3 caches reduce memory latency

6. TLB: - Cache page table entries - Speed up address translation

7. Memory Pooling: - Pre-allocate memory pools - Reduce allocation overhead

---

★Memory protection mechanism

Goal: Prevent process from accessing unauthorized memory.

Mechanisms:

1. Base and Limit Registers:

```
Base Register = 300040
Limit Register = 120900

Legal addresses: 300040 to 420940

If process tries to access 500000:
→ 500000 > (Base + Limit)
→ TRAP to OS (segmentation fault)
```

2. Page Table Protection Bits:

Each page table entry has **protection bits:**

Frame Number	Valid	Read	Write

- **Valid bit:** Page belongs to process's address space
- **Read/Write bits:** Access permissions

Access violation: CPU traps to OS → terminate process.

3. Segmentation: - Segments have separate base/limit - Each segment has protection (code=read-only, data=read-write)

★External fragmentation solution. Segmentation hardware.

External Fragmentation Solution:

1. Compaction: - Shuffle processes to combine free holes - Expensive (requires relocating processes) - Only if relocation dynamic

2. Paging: - Eliminate variable-sized partitions - Use fixed-size pages/frames - **No external fragmentation**

3. Segmentation with Paging: - Combine benefits of both

Segmentation Hardware:

Segment Table:

Segment	Base Addr	Limit
0	1400	1000
1	6300	400
2	4300	400

Logical Address:

Segment Num	Offset

Translation: 1. Use segment number to index segment table 2. Check: offset < limit? (else trap) 3. Physical address = base + offset

Example:

```
Logical: (Segment 2, Offset 53)
→ Segment table: Base = 4300, Limit = 400
```

```
→ 53 < 400? Yes
→ Physical = 4300 + 53 = 4353
```

★Calculate EAT with TLB

Problem: Hit ratio = 80%, Memory time = 200 ns, TLB time = 20 ns

Formula:

```
EAT = α × (T_tlb + T_mem) + (1-α) × (T_tlb + 2×T_mem)

where α = hit ratio
```

Calculation:

```
EAT = 0.8 × (20 + 200) + 0.2 × (20 + 400)
    = 0.8 × 220 + 0.2 × 420
    = 176 + 84
    = 260 ns
```

Answer: 260 nanoseconds

Interpretation: - 80% of time: TLB hit → 1 memory access (220 ns) - 20% of time: TLB miss → 2 memory accesses (420 ns) - Average: 260 ns

---

END OF DOCUMENT

Complete OS Exam Preparation — All Questions Answered
Based on Operating System Concepts, 10th Edition — Silberschatz, Galvin & Gagne